

LAB 2

Containerization & Kubernetes Orchestration

Task 4 — Docker Network

Container-to-container communication using names and IP addresses

Student	Kunal Prajapati
Custom network	lab2-network
Network driver	bridge
Containers	network-server and network-client

This report documents both communication methods required by Task 4: communication through the Docker container name and communication through the container IP address.

1. Create and Verify the Docker Network

The custom bridge network **lab2-network** was created using **docker network create lab2-network**. The subsequent **docker network ls** output shows **lab2-network** with the **bridge** driver and local scope.

```
PS D:\Desktop\lab2-task1> docker network create lab2-network
659380b98fa2cb0198b3c6b8bad29fbb38d1531d61730595aa900cf0f66649af
PS D:\Desktop\lab2-task1> docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
7f3266301da1        bridge              bridge              local
959ecbe1550a        host                host                local
659380b98fa2        lab2-network        bridge              local
f3b02231a660        lab2-task1_default  bridge              local
ddbb097e9665        none                null                local
PS D:\Desktop\lab2-task1>
```

Figure 1. Custom Docker network lab2-network is created and listed with the bridge driver.

This establishes an isolated user-defined Docker network on which the two test containers can communicate.

2. Communication Using Container Names

A container named **network-server** was started on **lab2-network**. A second container named **network-client** was also attached to the same network. The server container was verified independently by creating and reading **/server-message.txt**.

```
PS D:\Desktop\lab2-task1> docker run -dit --name network-server --network lab2-network alpine sh
bf82eff43fd9e5e9a186bf4e279e14202c0761225d2d18953c78ce727fdc08b6
PS D:\Desktop\lab2-task1> docker exec network-server sh -c "echo 'Hello from network-server' > /server-message.txt"
PS D:\Desktop\lab2-task1> docker exec network-server cat /server-message.txt
Hello from network-server
PS D:\Desktop\lab2-task1>
```

Figure 2. network-server is connected to lab2-network and a test message is created and read inside the container.

The client then executed **ping -c 3 network-server**. Docker's user-defined network DNS resolved the container name **network-server** to **172.19.0.2**. All three packets were received, giving **0% packet loss**.

```
PS D:\Desktop\lab2-task1> docker run -dit --name network-client --network lab2-network alpine sh
9ed39c58c85ba10426164b93b1eba9a5c4f769a5d6bd626d920f984fe3a39012
PS D:\Desktop\lab2-task1> docker exec network-client ping -c 3 network-server
PING network-server (172.19.0.2): 56 data bytes
64 bytes from 172.19.0.2: seq=0 ttl=64 time=0.457 ms
64 bytes from 172.19.0.2: seq=1 ttl=64 time=0.164 ms
64 bytes from 172.19.0.2: seq=2 ttl=64 time=0.208 ms

--- network-server ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.164/0.276/0.457 ms
PS D:\Desktop\lab2-task1>
```

Figure 3. network-client communicates with network-server by container name; three ping replies are received with 0% packet loss.

Result: communication using the container name **network-server** was successful.

3. Communication Using Container IP Address

The server's IP address was obtained with `docker inspect -f "{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}" network-server`. The output returned **172.19.0.2**.

```
PS D:\Desktop\lab2-task1> docker inspect -f "{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}" network-server
172.19.0.2
PS D:\Desktop\lab2-task1> docker exec network-client ping -c 3 172.19.0.2
PING 172.19.0.2 (172.19.0.2): 56 data bytes
64 bytes from 172.19.0.2: seq=0 ttl=64 time=0.410 ms
64 bytes from 172.19.0.2: seq=1 ttl=64 time=0.124 ms
64 bytes from 172.19.0.2: seq=2 ttl=64 time=0.147 ms

--- 172.19.0.2 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.124/0.227/0.410 ms
PS D:\Desktop\lab2-task1>
```

Figure 4. network-server IP address is inspected as 172.19.0.2 and the client successfully pings that IP with 0% packet loss.

The client then executed `ping -c 3 172.19.0.2`. All three packets were received with **0% packet loss**, confirming direct communication using the server container's IP address.

Result: communication using the container IP address **172.19.0.2** was successful.

4. Final Verification

Requirement	Evidence	Result
Create Docker network	lab2-network appears in docker network ls with bridge driver.	Passed
Name-based communication	network-client pinged network-server; 3/3 replies, 0% packet loss.	Passed
IP-based communication	network-server inspected as 172.19.0.2; client pinged that IP with 3/3 replies.	Passed

Both required communication methods are demonstrated successfully on the same user-defined Docker network. Name-based communication uses Docker's internal service/container-name resolution, while IP-based communication uses the inspected network IP.

Submission note: Upload this PDF in the Task 4 slot. Keep the Docker network commands and project/source files in the GitHub repository as supporting material.